

PART 1 : MCQ**Q1: Concept and Understanding (C2 Bloom's Taxonomy)****What is a stack in data structures?**

- A. A data structure that follows the FIFO (First In, First Out) principle
 B. A data structure that allows insertion and deletion from both ends ~~X~~
 C. A data structure that follows the LIFO (Last In, First Out) principle
 D. A data structure used for dynamic programming ?

Q2: Convert Infix to Prefix (C3 Bloom's Taxonomy)**Convert the infix expression $(A + B) * (C - D)$ to its prefix form.**

- A. $+AB*-CD$
 B. $*+AB-CD$
 C. $-+AB*CD$
 D. $AB+CD-*$

$$(A + B) * (C - D) \rightarrow$$

$$A + B \quad (C - D) \rightarrow *$$

$$A \ B \quad (C - D) \rightarrow * +$$

$$C - D \rightarrow * + AB$$

$$C D \rightarrow * + AB -$$

Q3: Push-Pop Operation and Stack Top (C4 Bloom's Taxonomy)

If the stack initially contains $[4, 7, 9]$ ($top = 9$) and the following operations are performed:

Push (6) ?
 Pop ()
 Push (3)

What is the top element of the stack now?

- A. 6
 B. 9
 C. 3
 D. 7

PART 2 : STRUCTURED**Instructions for Students**

For both parts of the structured question, **follow the steps of the algorithm as discussed/given in the lecture slides** to ensure accuracy and consistency.

Q1: Converting Infix to Postfix

Convert the following infix expression to postfix notation using the stack-based algorithm discussed in class. Complete the table below to show the steps.

Infix Expression:

$(15 * 3) / 9 - 20$

Table Format for Students to Fill

Step	Character Read (Ch)	Stack Content (operators)	Postfix Notation Created
1			
2			
3			
...			
Final			

Answer : _____ (Postfix Expression)

Q2 : Extend **the postfix expression** in Q1 using a stack-based approach. The evaluation follows the Simple Calculator algorithm where operands are pushed onto the stack, and operators act on the top two operands.

You also can use the table format based on lecture slide

Step	Postfix (Ch)	Operator/Operand	Stack Content	Result of Operation (if applicable)
1				
2				
3				
4				
5				
6				
7				

Answer : _____ (Result)

Q1 Converting infix to postfix

infix : $(15 * 3) / 9 - 20 \longrightarrow$ Postfix Expression : $15\ 3\ *\ 9\ /\ 20\ -$

Step	Character Read (ch)	Stack Content (operators)	Postfix Notation (created)
1	(	(	
2	15	(	15
3	*	(*	15
4	3	(*	15 3
5	)	(	15 3 *
6	/	(/	15 3 *
7	9	(/	15 3 * 9
8	-	(-	15 3 * 9 /
9	20	(-	15 3 * 9 / 20
10	end	empty	15 3 * 9 / 20 -

Q2 Extend the postfix expression

Step	Postfix (Ch)	Operator/Operand	Stack Content	Result of Operation (if applicable)
1	15	15	(none)	Push operand 15
2	3	15, 3	(none)	Push operand 3
3	*	45	15 * 3	Pop 15 and 3, push result 45
4	9	45, 9	(none)	Push operand 9
5	/	5	45 / 9	Pop 45 and 9, push result 5
6	20	5, 20	(none)	Push operand 20
7	-	-15	5 - 20	Pop 5 and 20, push result -15

Programming Question (Paired)

Task:

Write a C++ program that implements a stack to perform operations on a predefined list of characters. The program should:

1. **push a predefined set of characters** ({'A', 'B', 'C', 'D', 'E'}) onto the stack.
 2. **pop two characters** from the stack.
 3. **stackTop** to view the character at the top of the stack without removing it.
 4. **Check if the stack is empty** after the operations.
 5. **Display all remaining characters** in the stack.
-

This activity will be conducted in **pairs** to implement and compare two approaches for stack operations:

1. **Student A:** Implement the stack using an **array-based** approach.
2. **Student B:** Implement the stack using a **pointer-based (linked list)** approach.